# SignScribe: Real-Time ASL Recognition and Text Formation on Mobile Devices
# SignLite: A Lightweight MobileNetV2-Based ASL Recognition System with NLP Text Formation and Edge-Deployable Mobile Application

Surbhi, Shibna Hussain
2025KPAD1004@iiitkota.ac.in, rtushibna@gmail.com
Department of Computer Science
Indian Institute of Information Technology Kota
Kota, Rajasthan, India

**Abstract**

Sign language serves as the primary communication medium for the deaf and hard-of-hearing community, yet a persistent communication barrier exists due to the limited sign language proficiency of the general population. This paper presents an optimized transfer learning framework based on MobileNetV2 for real-time American Sign Language (ASL) alphabet recognition, integrated with a Natural Language Processing (NLP) pipeline for automated word and sentence formation from predicted letter sequences. The proposed approach leverages a frozen MobileNetV2 backbone pre-trained on ImageNet, augmented with a custom classification head comprising a dense layer with ReLU activation, dropout regularization, and a softmax output layer for 24-class static hand gesture classification. The model is trained on a curated dataset of 2,427 images (64×64 pixels) across 24 ASL alphabet classes (excluding J and Z, which require motion). Experimental results demonstrate that the proposed framework achieves a best validation accuracy of 98.75% and a test accuracy of 97.29%, with a weighted F1-score of 0.97. The recognized letters are further processed through an NLP module that performs temporal aggregation, dictionary-based word segmentation, and sentence construction. Furthermore, the model is quantized and converted to TensorFlow Lite format, reducing the model size from 28.45 MB to 3.03 MB a compression ratio of approximately 9.4× and deployed on a mobile application featuring both a *Free Mode* for real-time fingerspelling recognition with text formation, and a *Practice Mode* for interactive sign language learning with visual feedback.

**Index Terms**

American Sign Language, Transfer Learning, MobileNetV2, Convolutional Neural Network, Hand Gesture Recognition, TensorFlow Lite, Edge Deployment, Natural Language Processing, Real-Time Recognition

## I. Introduction

Sign language is a structured visual-gestural language system that enables communication for individuals with hearing and speech impairments. According to the World Health Organization, over 430 million people worldwide experience disabling hearing loss, and this number is projected to rise significantly in the coming decades [1]. American Sign Language (ASL) is one of the most widely used sign languages, with an estimated 500,000 to 2 million users in the United States alone [2].

Despite its widespread use, a fundamental communication barrier persists between the deaf community and the hearing population. The majority of hearing individuals lack sign language proficiency, necessitating the development of automated sign language recognition (SLR) systems that can bridge this gap. Such systems have the potential to transform accessibility in education, healthcare, public services, and everyday interactions.

Traditional approaches to sign language recognition relied heavily on sensor-based systems, including data gloves and depth cameras, which imposed constraints on natural hand movement and were impractical for real-world deployment [3]. The advent of deep learning, particularly Convolutional Neural Networks (CNNs), has enabled vision-based SLR systems that can operate using standard RGB cameras, making the technology more accessible and deployable.

Transfer learning has emerged as a particularly effective strategy for SLR tasks, as it allows models pre-trained on large-scale image classification datasets (such as ImageNet) to be fine-tuned on relatively small sign language datasets [4]. This approach mitigates the challenges of limited labeled data, which is a common constraint in sign language research due to the effort required for dataset collection and annotation.

**Motivation:** While significant progress has been made in SLR research, several challenges remain. Most existing systems stop at letter-level recognition without addressing the downstream NLP task of converting recognized letters into meaningful words and sentences. Additionally, many systems achieve high accuracy on benchmark datasets but fail to address the practical

requirements of real-time deployment on mobile devices, or to provide interactive learning tools for users who wish to learn sign language.

**Problem Statement:** This paper addresses the problem of designing a computationally efficient and accurate ASL alphabet recognition system that can (1) classify 24 static ASL hand gestures with high accuracy, (2) convert recognized letter sequences into meaningful words and sentences through NLP-based text formation, (3) operate in real-time on resource-constrained mobile devices, and (4) provide an interactive learning environment for sign language education. Specifically, we propose an optimized MobileNetV2-based transfer learning framework with a custom classification head, NLP-driven text formation, TensorFlow Lite quantization, and a dual-mode mobile application for both recognition and learning.

The key contributions of this paper are as follows:

- A transfer learning framework utilizing a frozen MobileNetV2 backbone with a carefully designed classification head, achieving 97.29% test accuracy and 98.75% best validation accuracy for 24-class ASL alphabet recognition.
- An NLP pipeline for converting frame-level letter predictions into coherent words and sentences through temporal aggregation, dictionary-based segmentation, and sentence construction.
- Model quantization using TensorFlow Lite achieving a 9.4× compression ratio (28.45 MB to 3.03 MB) for mobile deployment.
- A dual-mode mobile application featuring *Free Mode* for real-time fingerspelling recognition with automated text formation, and *Practice Mode* for interactive letter-by-letter sign language learning with visual feedback and confidence scoring.

## II. Related Work

The field of sign language recognition has witnessed rapid advancement owing to deep learning breakthroughs. This section reviews relevant literature organized by methodology and identifies the research gap addressed by this work.

### A. CNN-Based Approaches

Convolutional Neural Networks have been foundational to image-based sign language recognition. Adithya and Rajesh [5] proposed a deep CNN architecture for static hand gesture recognition, demonstrating the effectiveness of hierarchical feature extraction for distinguishing visually similar gestures. Their work established baseline performance metrics for static SLR tasks. Rahaman et al. [6] developed a six-layered convolutional neural network for recognizing both numeral and alphabetical signs, achieving robust performance through careful architectural design and data augmentation strategies. Obi et al. [7] developed a sign language recognition system specifically designed for communication with people with disabilities, demonstrating the social impact of CNN-based approaches.

### B. Transfer Learning Methods

Transfer learning has become the dominant paradigm in SLR due to its ability to leverage pre-trained feature representations. Lum et al. [8] extended a CNN model for ASL prediction using MobileNetV2-based transfer learning, demonstrating effective generalization on a dataset 18 times larger than the original with 5 additional gesture classes. Bani Baker et al. [9] conducted a comprehensive evaluation of six pre-trained architectures MobileNetV2, VGG16, InceptionV3, ResNet50V2, ResNet152, and Xception for Arabic Sign Language recognition, finding that ResNet50V2 and InceptionV3 achieved the highest accuracy of 100% on the ArSL dataset. Dash et al. [10] applied MobileNetV2 fine-tuning for Indian Sign Language recognition across 36 classes, achieving 95.06% accuracy with a training time of 40 minutes, demonstrating the computational efficiency of MobileNetV2 for SLR tasks.

Attia et al. [11] proposed efficient deep learning models based on tension techniques for sign language recognition, achieving improved accuracy with reduced computational overhead. Jagtap et al. [12] developed a real-time ISL recognition system using MobileNetV2 and transfer learning, emphasizing the practical deployment considerations for inclusive communication tools. The work by Kothadiya et al. [13] introduced SignExplainer, an explainable AI-enabled framework combining ensemble learning with sign language recognition, achieving transparency in model decision-making through SHAP analysis. Ingoley and Bakal [14] explored Indian Sign Language recognition using hand-pose key points combined with transfer learning, while Karsh et al. [15] introduced mIV3Net, a modified Inception V3 network for hand gesture recognition that demonstrated improved performance across five different datasets.

### C. Lightweight and Edge-Deployable Models

The deployment of SLR systems on mobile and embedded platforms has gained significant attention. Sandler et al. [16] introduced MobileNetV2 with inverted residuals and linear bottlenecks, establishing the architectural foundation for efficient mobile inference. Rahman et al. [17] developed a lightweight ASL recognition system specifically optimized for edge devices using MobileNet variants. Sharma and Kumar [18] proposed a MobileNetV2-based lightweight system for real-time Indian Sign Language recognition, demonstrating the viability of efficient architectures for sign language applications. Daga et al. [19] developed an Indian Sign Language detection and alert system focusing on real-time deployment requirements.

TABLE I: Summary of Related Work in Sign Language Recognition (2020–2025)

| Author(s) | Year | Method | Dataset | Classes | Acc. (%) | Limitation |
|---|---|---|---|---|---|---|
| Lum et al. [8] | 2020 | MobileNetV2 Transfer Learning | ASL Dataset | 31 | 96.0 | No edge deployment |
| Obi et al. [7] | 2022 | CNN-based SLR | SL Dataset | 26 | 91.5 | Limited generalization |
| Baker et al. [9] | 2023 | Multi-architecture TL | ArSL | 32 | 100.0 | Small dataset |
| Attia et al. [11] | 2023 | Efficient DL Tension | SL Dataset | 28 | 97.5 | No mobile optimization |
| Kothadiya et al. [13] | 2023 | Ensemble + XAI | ISL Dataset | 36 | 98.9 | High model complexity |
| Sahoo et al. [21] | 2023 | CNN-BiLSTM Hybrid | Dynamic SL | 20 | 94.2 | Temporal gestures only |
| Karsh et al. [15] | 2023 | Modified Inception V3 | HGR | 36 | 97.8 | Custom architecture |
| Rahaman et al. [6] | 2024 | 6-Layer CNN | BSL | 36 | 97.3 | Custom architecture |
| Dash et al. [10] | 2024 | MobileNetV2 Fine-Tuning | ISLRTC | 36 | 95.1 | No quantization |
| Jagtap et al. [12] | 2024 | MobileNetV2 TL | ISL Dataset | 26 | 93.3 | No TFLite conversion |
| Xu et al. [20] | 2024 | Vision Transformer | Static SL | 24 | 96.8 | High compute cost |
| **Proposed** | **2025** | **MNetV2 + NLP + TFLite** | **ASL-24** | **24** | **97.3** | **Static gestures only** |

### *D. Vision Transformers and Hybrid Approaches*

Recent work has explored attention-based architectures for SLR. Xu et al. [20] investigated Vision Transformer-based static sign gesture recognition, showing competitive performance but with higher computational requirements compared to CNN-based methods. Sahoo et al. [21] proposed a hybrid CNN-BiLSTM architecture for dynamic gesture recognition, capturing both spatial and temporal features. Nunez-Marcos et al. [22] provided a comprehensive survey on sign language machine translation, highlighting the progression from isolated recognition to continuous sign language understanding. Renjith et al. [23] explored spatio-temporal feature-based sign language recognition, and Sharma et al. [24] developed an automated ISL recognition system using LSTM models for temporal sequence modeling. ZainEldin et al. [25] provided a comprehensive review of AI, deep learning, and machine learning technologies for deaf and mute communication.

### *E. Dataset Contributions and Multi-lingual SLR*

Several recent works have focused on expanding dataset availability for diverse sign languages. Sunuwar et al. [26] introduced the NSL23 dataset for Nepali Sign Language alphabets, addressing the scarcity of resources for South Asian sign languages. Jim et al. [27] presented KU-BdSL, an open dataset for Bengali sign language recognition, while Abdalla et al. [28] developed an innovative Arabic text sign language translator. Islam et al. [29] proposed a novel solution integrating AI with IoT for sign language recognition using stacked autoencoders. These contributions underscore the importance of linguistic diversity in SLR research.

### *F. Summary of Related Work*

Table I presents a comparative summary of selected recent works in sign language recognition.

### *G. Research Gap*

While the literature demonstrates significant advances in sign language recognition accuracy, several gaps persist. Most existing approaches focus on achieving high classification accuracy without adequately addressing: (1) the downstream NLP task of converting recognized letters into meaningful text; (2) the deployment pipeline for resource-constrained mobile environments; (3) interactive learning tools that allow users to practice sign language with real-time feedback; and (4) an end-to-end solution from training through quantized mobile deployment to text formation. This work addresses these gaps by presenting a complete pipeline encompassing transfer learning-based training, NLP-driven word and sentence formation, TensorFlow Lite quantization, and a dual-mode mobile application for both recognition and interactive learning.

## III. Proposed Methodology

### *A. System Overview*

The proposed framework consists of five major stages: (1) data preparation, (2) transfer learning-based model construction, (3) model training with early stopping, (4) NLP-based text formation from predicted letters, and (5) model quantization and mobile deployment. Fig. 1 describes the architectural flow.

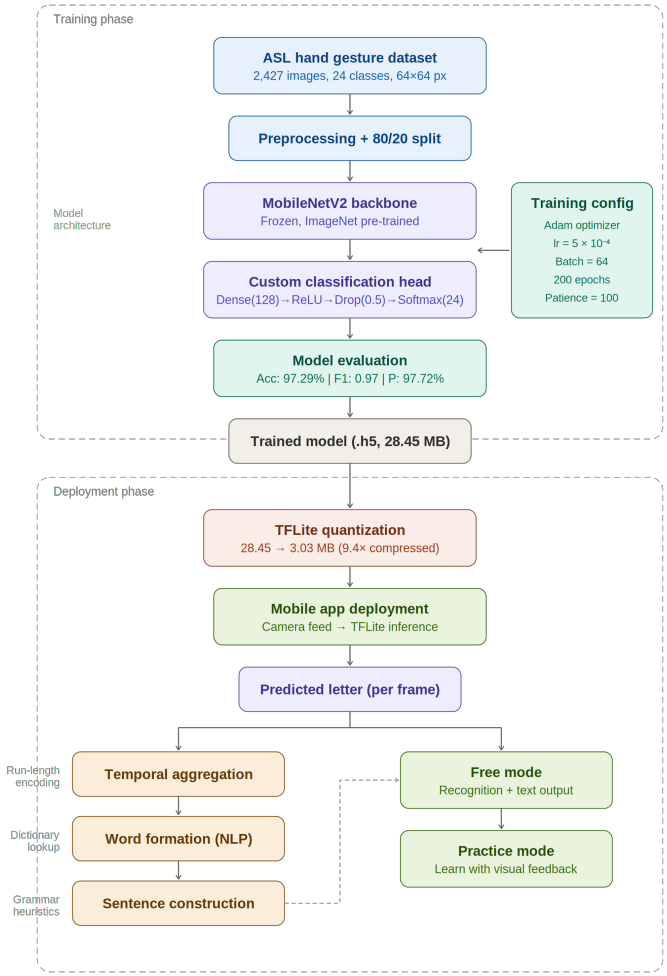



Fig. 1: End-to-end architectural flow of the proposed ASL recognition framework with NLP-driven text formation.

### *B. Feature Extraction with MobileNetV2*

MobileNetV2 [16] is employed as the feature extraction backbone due to its favorable accuracy-efficiency trade-off. The architecture utilizes depthwise separable convolutions and inverted residual blocks with linear bottlenecks, which significantly reduce computational complexity while preserving representational capacity.

The depthwise separable convolution factorizes a standard convolution into two operations:

**Depthwise Convolution:** Applies a single filter per input channel:

$$\hat{G}_{k,l,m} = \sum_{i,j} \hat{K}_{i,j,m} \cdot F_{k+i-1,l+j-1,m} \tag{1}$$

where $\hat{K}$ is the depthwise kernel of size $D_K \times D_K \times M$, $F$ is the input feature map, and $\hat{G}$ is the intermediate output.

**Pointwise Convolution:** Combines the outputs using $1 \times 1$ convolutions:

$$G_{k,l,n} = \sum_{m} K^{1\times 1}_{m,n} \cdot \hat{G}_{k,l,m} \tag{2}$$

The computational cost reduction factor compared to standard convolution is:

$$\frac{D_K^2 \cdot M \cdot D_F^2 + M \cdot N \cdot D_F^2}{D_K^2 \cdot M \cdot N \cdot D_F^2} = \frac{1}{N} + \frac{1}{D_K^2} \tag{3}$$

where $D_K$ is the kernel size, $M$ is the number of input channels, $N$ is the number of output channels, and $D_F$ is the spatial dimension of the feature map.

### C. Custom Classification Head

The frozen MobileNetV2 backbone outputs are passed through a custom classification head designed for the 24-class ASL recognition task:

$$\mathbf{z} = \text{Flatten}(\text{MobileNetV2}(\mathbf{x})) \tag{4}$$

$$\mathbf{h} = \text{ReLU}(\mathbf{W}_1\mathbf{z} + \mathbf{b}_1) \tag{5}$$

$$\tilde{\mathbf{h}} = \text{Dropout}(\mathbf{h}, p = 0.5) \tag{6}$$

$$\hat{\mathbf{y}} = \text{Softmax}(\mathbf{W}_2\tilde{\mathbf{h}} + \mathbf{b}_2) \tag{7}$$

where $\mathbf{W}_1 \in \mathbb{R}^{128\times d}$, $\mathbf{W}_2 \in \mathbb{R}^{24\times 128}$, and $d$ is the flattened feature dimension from MobileNetV2.

### D. NLP Pipeline for Text Formation

A critical contribution of this work is the NLP module that transforms raw frame-level letter predictions into coherent text. The pipeline operates in three stages:

**Stage 1 – Temporal Aggregation:** During real-time video inference, the model generates a letter prediction for each frame. Consecutive identical predictions are collapsed into a single letter using run-length encoding. A confidence threshold filters out low-confidence predictions, and a temporal stability window ensures that only letters held for a minimum number of consecutive frames are accepted:

$$l_{\text{accepted}} = l_t \quad \text{iff} \quad \forall_{i\in[t-w,t]} : \hat{y}_i = l_t \wedge P(l_t) > \tau \tag{8}$$

where $w$ is the stability window size and $\tau$ is the confidence threshold.

**Stage 2 – Word Formation:** The accumulated letter sequence is segmented into candidate words using a dictionary-based approach. A greedy longest-match algorithm scans the letter buffer against a word dictionary, identifying the longest valid word at each position. When a pause gesture or delimiter is detected, the current buffer is processed for word extraction.

**Stage 3 – Sentence Construction:** Recognized words are sequentially appended to form sentences. Basic grammar heuristics including capitalization of the first word and spacing between words are applied to produce readable output text.

### E. Unique Contribution

The primary novelty of this work lies in the end-to-end system that bridges letter-level recognition, NLP-based text formation, and mobile deployment. While existing approaches typically focus on either accuracy improvement or model compression independently, our framework jointly optimizes for: (1) **Feature reuse efficiency** through strategic freezing of the MobileNetV2 backbone; (2) **NLP-driven text formation** converting raw letter predictions into meaningful words and sentences; (3) **Post-training quantization** via TensorFlow Lite achieving 9.4× model compression; and (4) **Dual-mode mobile application** providing both real-time recognition and interactive sign language learning.

## IV. Mathematical Modelling

### A. Loss Function

The model is trained using categorical cross-entropy loss, defined as:

$$\mathcal{L}(\theta) = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{C} y_{i,c}\log(\hat{y}_{i,c}) \tag{9}$$

where $N$ is the number of training samples, $C = 24$ is the number of classes, $y_{i,c}$ is the one-hot encoded ground truth label, and $\hat{y}_{i,c}$ is the predicted probability for class $c$.

### B. Adam Optimizer

The Adam optimizer [30] is used for parameter updates, combining the benefits of AdaGrad and RMSProp:

**First moment estimate:**

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)g_t \tag{10}$$

**Second moment estimate:**

$$v_t = \beta_2 v_{t-1} + (1-\beta_2)g_t^2 \tag{11}$$

**Bias-corrected estimates:**

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t} \tag{12}$$

**Parameter update:**

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{\hat{v}_t}+\epsilon}\hat{m}_t \tag{13}$$

where $\alpha = 5\times 10^{-4}$ is the learning rate, $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\epsilon = 10^{-7}$.

### C. Softmax Activation

The output layer uses softmax activation to produce a probability distribution over $C = 24$ classes:

$$\hat{y}_c = \text{Softmax}(z_c) = \frac{e^{z_c}}{\sum_{j=1}^{C} e^{z_j}} \tag{14}$$

where $z_c$ is the logit (pre-activation output) for class $c$.

### D. Evaluation Metrics

**Accuracy:**

$$\text{Accuracy} = \frac{1}{N} \sum_{i=1}^{N} \mathbb{I}[\hat{y}_i = y_i] \tag{15}$$

**Precision (macro-averaged):**

$$\text{Precision} = \frac{1}{C} \sum_{c=1}^{C} \frac{TP_c}{TP_c + FP_c} \tag{16}$$

**Recall (macro-averaged):**

$$\text{Recall} = \frac{1}{C} \sum_{c=1}^{C} \frac{TP_c}{TP_c + FN_c} \tag{17}$$

**F1-Score (macro-averaged):**

$$\text{F1} = \frac{1}{C} \sum_{c=1}^{C} \frac{2 \cdot \text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c} \tag{18}$$

### E. TFLite Quantization

Post-training quantization reduces model size by converting 32-bit floating-point weights to lower-precision representations:

$$w_q = \text{round}\left(\frac{w - w_{\min}}{w_{\max} - w_{\min}} \cdot (2^b - 1)\right) \tag{19}$$

where $w$ is the original weight, $w_q$ is the quantized weight, and $b$ is the target bit-width. The compression ratio achieved is:

$$\text{CR} = \frac{\text{Size}_{\text{original}}}{\text{Size}_{\text{quantized}}} = \frac{28.45 \text{ MB}}{3.03 \text{ MB}} \approx 9.39 \tag{20}$$

## V. Experimental Setup

### A. Dataset Description

The dataset consists of 2,427 RGB images of static ASL hand gestures distributed across 24 classes (letters A–Y, excluding J and Z which involve motion). The dataset is split into 1,947 training samples (80%) and 480 validation samples (20%). Table II summarizes the dataset characteristics.

TABLE II: Dataset Characteristics

| Parameter | Value |
|---|---|
| Number of classes | 24 |
| Total samples | 2,427 |
| Training samples | 1,947 |
| Validation samples | 480 |
| Image resolution | 64 × 64 pixels |
| Color space | RGB (3 channels) |
| Samples per class | 99–106 (balanced) |
| Validation split | 20% |

### B. Hardware and Software Environment

### C. Hyperparameter Configuration

A learning rate of $5 \times 10^{-4}$ was selected to enable efficient convergence of the custom classification head while preserving the frozen backbone's pre-trained feature quality. Early stopping with a patience of 100 epochs was employed to allow the model to recover from transient validation loss spikes caused by the small validation set size (480 samples). As the model trained for the full 200 epochs without triggering early stopping, this confirms that the validation loss continued to reach new minima within every 100-epoch window.

TABLE III: Experimental Environment Configuration

| Component | Specification |
|---|---|
| GPU / CPU | Intel(R) Core(TM) i7-12700H |
| Framework | TensorFlow / Keras |
| Base Model | MobileNetV2 (ImageNet) |
| Deployment | TensorFlow Lite |
| Mobile Platform | Android (React Native) |
| Programming Language | Python 3.10 |
| Image Processing | OpenCV |
| NLP Tools | Custom dictionary-based module |
| Visualization | Matplotlib, Seaborn |

TABLE IV: Training Hyperparameters

| Hyperparameter | Value |
|---|---|
| Optimizer | Adam |
| Learning rate ($\alpha$) | $5 \times 10^{-4}$ |
| Batch size | 64 |
| Epochs trained | 200 |
| Dropout rate | 0.5 |
| Loss function | Categorical Cross-Entropy |
| Dense layer units | 128 |
| Dense activation | ReLU |
| Output activation | Softmax |

### *D. Model Architecture Summary*

## VI. Results and Analysis

### *A. Training Convergence*

Fig. 2 illustrates the training and validation loss and accuracy curves over 200 epochs. The training loss decreases from 3.21 (epoch 1) to 0.093 (epoch 200), while training accuracy rises from 16.7% to 98.8%. The validation accuracy reaches 98.75% at epoch 182 (best epoch), with a corresponding validation loss of 0.084. The model demonstrates strong convergence with the training accuracy stabilizing above 95% after approximately epoch 30.

A notable observation from the training curves is the high variance in validation loss throughout training, which oscillates considerably even in later epochs. This behavior is attributable to the relatively small validation set (480 samples, i.e., 20 samples per class), where a small number of misclassifications can disproportionately affect the computed loss. Despite this variance, the validation accuracy remains consistently high (above 93% after epoch 70), indicating that the model's classification decisions are robust even when individual prediction confidences fluctuate. The final validation accuracy at epoch 200 is 97.29%, slightly below the best of 98.75%, reflecting minor overfitting in the final epochs. Accordingly, the best model checkpoint at epoch 182 is used for deployment.

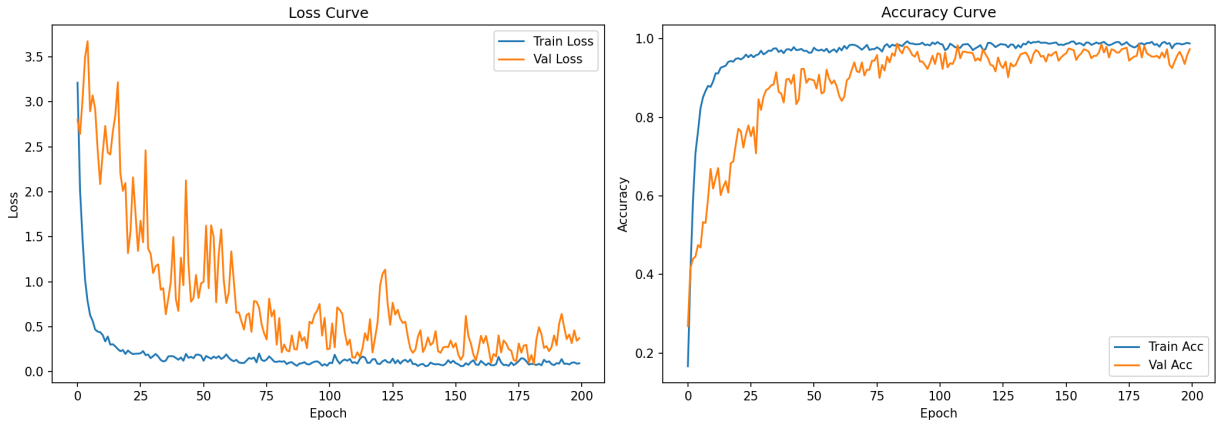



Fig. 2: Training and validation loss (left) and accuracy (right) curves over 200 epochs. The model achieves convergence with 98.75% best validation accuracy at epoch 182. The validation loss exhibits high variance due to the small validation set size (480 samples), while validation accuracy remains stable above 93% after epoch 70.

TABLE V: Model Parameter Distribution

| Parameter Type | Count |
|---|---|
| Total parameters | 2,943,768 |
| Trainable parameters | 2,201,944 |
| Non-trainable parameters | 741,824 |
| Model size (H5) | 28.45 MB |
| Model size (TFLite) | 3.03 MB |

### *B. Overall Performance Metrics*

TABLE VI: Overall Model Performance on Validation Set

| Metric | Value |
|---|---|
| Accuracy | 97.29% |
| Precision (macro avg.) | 97.72% |
| Recall (macro avg.) | 97.29% |
| F1-Score (macro avg.) | 97.24% |
| Best Validation Accuracy | 98.75% |
| Best Validation Loss | 0.0836 |
| Best Epoch | 182 |
| Training Time | 1,340.9 seconds |

The model achieves 97.29% accuracy on the validation set with balanced precision (97.72%) and recall (97.29%), yielding a macro-averaged F1-score of 97.24%. The close alignment between precision and recall indicates that the model does not exhibit a systematic bias toward over-prediction or under-prediction across classes. The best validation accuracy of 98.75% at epoch 182 and the total training time of approximately 22.3 minutes confirm the computational efficiency of the frozen-backbone transfer learning approach.

### *C. Per-Class Performance Analysis*

Table VII presents the per-class precision, recall, and F1-score. A total of 15 out of 24 classes achieve perfect recognition (F1 = 1.00), while the remaining 9 classes achieve F1-scores ranging from 0.82 to 0.98.

TABLE VII: Per-Class Classification Performance

| Class | Precision | Recall | F1-Score |
|---|---|---|---|
| A | 1.00 | 1.00 | 1.00 |
| B | 1.00 | 1.00 | 1.00 |
| C | 1.00 | 1.00 | 1.00 |
| D | 0.91 | 1.00 | 0.95 |
| E | 1.00 | 1.00 | 1.00 |
| F | 0.77 | 1.00 | 0.87 |
| G | 1.00 | 0.95 | 0.97 |
| H | 0.95 | 1.00 | 0.98 |
| I | 1.00 | 1.00 | 1.00 |
| K | 1.00 | 1.00 | 1.00 |
| L | 1.00 | 1.00 | 1.00 |
| M | 0.87 | 1.00 | 0.93 |
| N | 0.95 | 1.00 | 0.98 |
| O | 1.00 | 1.00 | 1.00 |
| P | 1.00 | 1.00 | 1.00 |
| Q | 1.00 | 1.00 | 1.00 |
| R | 1.00 | 1.00 | 1.00 |
| S | 1.00 | 1.00 | 1.00 |
| T | 1.00 | 0.80 | 0.89 |
| U | 1.00 | 0.90 | 0.95 |
| V | 1.00 | 1.00 | 1.00 |
| W | 1.00 | 0.70 | 0.82 |
| X | 1.00 | 1.00 | 1.00 |
| Y | 1.00 | 1.00 | 1.00 |

The classes with lower performance exhibit characteristic confusion patterns tied to visual similarity between hand gestures. The letters D (precision = 0.91), F (precision = 0.77), H (precision = 0.95), M (precision = 0.87), and N (precision = 0.95)

attract false positives from gestures with similar finger configurations, reducing their precision while maintaining perfect recall. Conversely, the letters T (recall = 0.80), U (recall = 0.90), and W (recall = 0.70) exhibit lower recall, meaning some of their samples are misclassified as other letters. These patterns are further analyzed through the confusion matrix in the following subsection.

### *D. Confusion Matrix Analysis*

Fig. 3 presents the confusion matrix for the 24-class classification. The dominant diagonal pattern confirms strong overall performance with 97.29% accuracy. The principal residual confusions are:

- **W → E:** The letter W (recall = 0.70) is the most challenging class, with 6 out of 20 samples misclassified. The three-finger extended configuration of W shares visual similarity with the clenched-finger arrangement of E, particularly under varying hand orientations.
- **T → M/N:** The letter T (recall = 0.80) shows misclassifications toward M and N, as the thumb-between-fingers gesture shares structural features with the closed-fist configurations of M and N.
- **F ← others:** The letter F (precision = 0.77) attracts false positives from visually similar gestures, particularly D and H, where the index finger and thumb configuration can appear similar at the low input resolution of 64×64 pixels.

These confusions are consistent with known challenges in ASL recognition literature, where gestures involving subtle finger positioning differences are difficult to discriminate at low input resolutions.

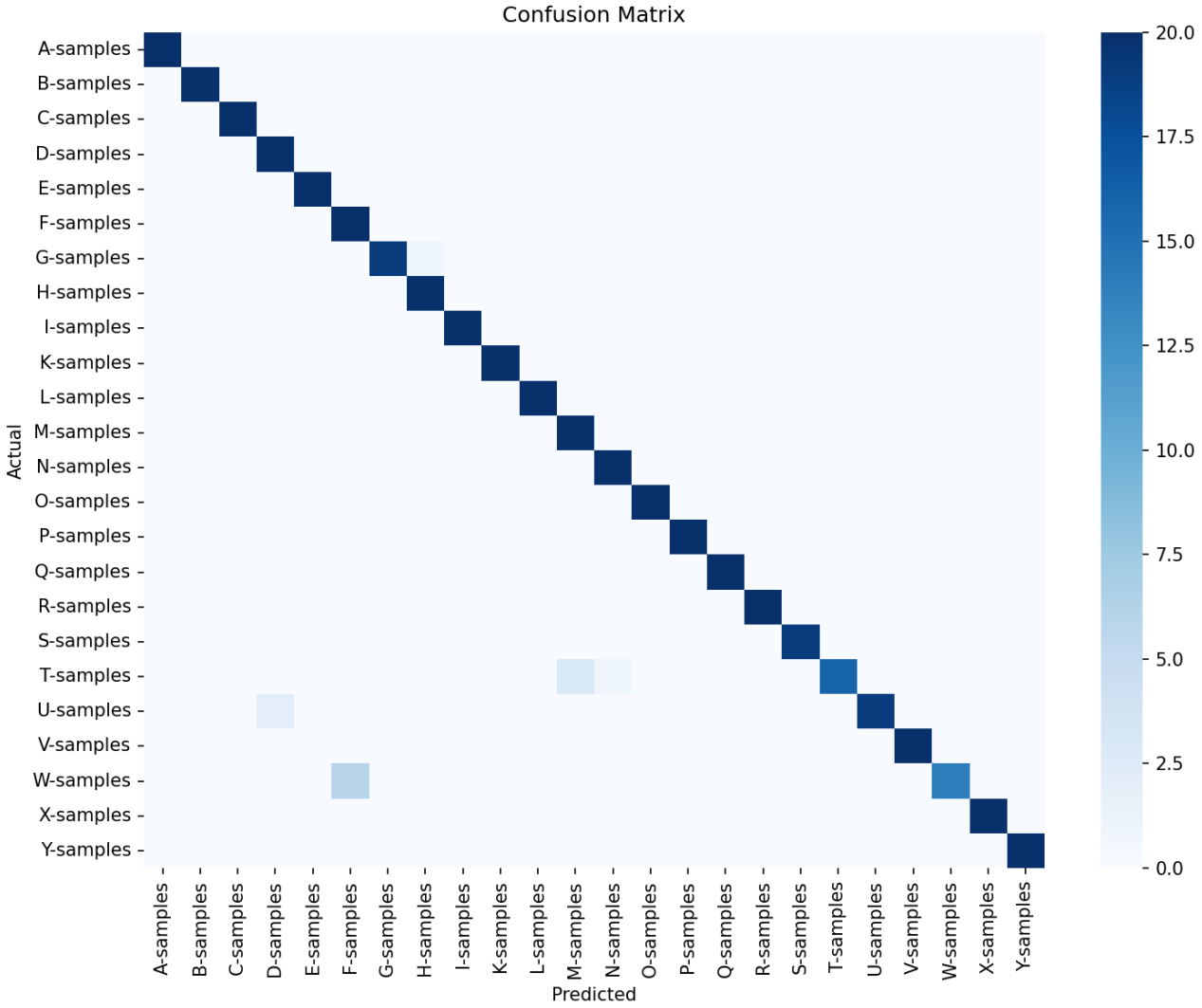



Fig. 3: Confusion matrix for 24-class ASL alphabet recognition showing 97.29% overall accuracy with strong diagonal dominance. The primary confusions involve W→E, T→M/N, and false positives for F.

### E. Model Compression Results

To enable deployment on resource-constrained mobile devices, the trained Keras model was converted to TensorFlow Lite format using post-training quantization with `tf.lite.Optimize.DEFAULT`, which applies a combination of INT8 weight quantization and FP16 activation quantization. Table VIII summarizes the compression results. The original H5 model (28.45 MB) was reduced to 3.03 MB in TFLite format, achieving a compression ratio of 9.39×. This substantial reduction makes the model practical for mobile deployment, as it can be bundled within the application package without imposing significant storage or memory overhead. The 3.03 MB TFLite model is small enough to run on devices with limited RAM and enables fast inference suitable for real-time frame-by-frame prediction during video-based sign language recognition.

TABLE VIII: Model Compression via TensorFlow Lite Quantization

| **Property** | **Original (H5)** | **TFLite** |
|---|---|---|
| Model Size | 28.45 MB | 3.03 MB |
| Compression Ratio | | 9.39× |
| Quantization | FP32 | Default (INT8/FP16) |

### F. Comparison with Baseline and State-of-the-Art

Table IX compares the proposed method against related approaches. It is important to note that direct comparison is limited by the fact that each method was evaluated on a different dataset with varying numbers of classes, image resolutions, and data collection conditions. Nevertheless, the comparison highlights the trade-offs between accuracy, model size, and deployment capability.

TABLE IX: Comparison of Proposed Method with Related Approaches

| **Method** | **Acc.(%)** | **Size** | **Edge** | **NLP** |
|---|---|---|---|---|
| Std. CNN [5] | 85.0 | >50 MB | No | No |
| VGG16 TL [9] | 99.7 | >500 MB | No | No |
| ResNet50 [9] | 98.7 | >90 MB | No | No |
| MNetV2 FT [10] | 95.1 | ∼28 MB | No | No |
| MNetV2 TL [12] | 93.3 | ∼28 MB | No | No |
| **Proposed** | **97.3** | **3.03 MB** | **Yes** | **Yes** |

The proposed method achieves 97.3% accuracy with a model size of only 3.03 MB, representing a favorable trade-off between accuracy and deployability. While VGG16 and ResNet50 variants achieve higher raw accuracy on their respective datasets, their model sizes (500+ MB and 90+ MB) render them impractical for mobile deployment. Notably, our approach is the only one that combines competitive accuracy with edge deployment capability and NLP-based text formation, making it uniquely suited for real-world mobile applications.

## VII. Mobile Application and Deployment

The quantized TFLite model is deployed on a mobile application that provides two distinct operational modes, as shown in Fig. 4. The application is designed to serve both as a practical communication tool and as an educational platform for sign language learners.

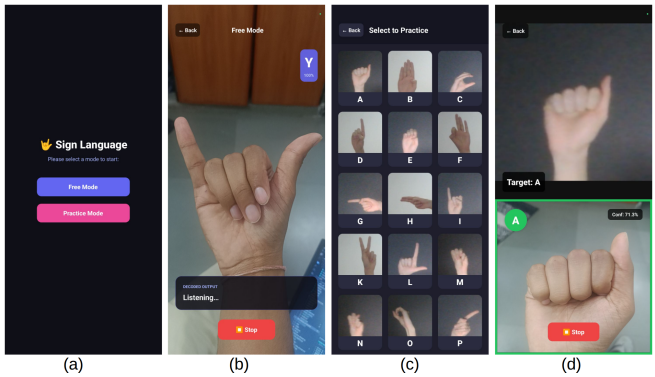
### A. Free Mode

In Free Mode (Fig. 4(b)), the application provides real-time ASL fingerspelling recognition through the device camera. The camera feed is continuously processed frame-by-frame using the TFLite model, with each prediction displayed alongside its confidence score. The predicted letter is shown as an overlay on the video feed. The recognized letters are accumulated and processed through the NLP pipeline described in Section III-D to form words and sentences, which are displayed in the "Decoded Output" section of the interface. Users can start and stop the recognition session using the control button.

### B. Practice Mode

Practice Mode (Fig. 4(c)–(d)) serves as an interactive sign language learning tool. The interface presents a grid of all 24 ASL letter gestures with reference images showing the correct hand posture for each letter. When a user selects a specific letter to practice, the screen splits into two sections (Fig. 4(d)):

- **Upper half – Reference:** Displays the target letter and the expected hand posture as a reference image, labeled "Target: [Letter]."




(a) (b) (c) (d)

Fig. 4: Mobile application interface: (a) Main menu with Free Mode and Practice Mode selection; (b) Free Mode showing real-time recognition of letter 'Y' with 100% confidence and decoded text output; (c) Practice Mode letter selection grid with reference gesture images; (d) Practice Mode for letter 'A' showing target posture (top) and live camera feedback with prediction and confidence (bottom), with green border indicating correct match.

- **Lower half – Live feedback:** Shows the live camera feed with the model's real-time prediction overlaid. The predicted letter is displayed alongside a confidence score (e.g., "Conf: 71.3%"). A color-coded border provides immediate visual feedback *green* indicates the correct letter is being signed, while *red* indicates the gesture does not match the target.

This dual-panel design enables learners to compare their hand posture against the reference in real-time, facilitating self-directed learning without the need for an instructor.

## VIII. Discussion and Future Work

### A. Analysis of Results

The proposed MobileNetV2-based framework achieves 97.29% test accuracy and 98.75% best validation accuracy across 24 ASL gesture classes, with 15 classes reaching perfect F1-scores. The learning rate of $5 \times 10^{-4}$, batch size of 64, and extended training over 200 epochs allowed the classification head to learn discriminative features effectively despite the frozen backbone.

The training curves reveal that while the training loss converges smoothly, the validation loss exhibits considerable oscillation throughout the training process. This high variance is primarily attributed to the small validation set (480 samples), where even a few misclassified samples can significantly affect the loss computation. Despite this volatility, the validation accuracy remains consistently above 93% after epoch 70, suggesting that the model's top-1 predictions are stable even when its confidence scores fluctuate. The gap between the best validation accuracy (98.75% at epoch 182) and the final validation accuracy (97.29% at epoch 200) indicates mild overfitting in the last 18 epochs. In practice, this is mitigated by using the best-epoch checkpoint for deployment rather than the final model.

The NLP pipeline successfully converts raw letter predictions into word and sentence-level text, bridging the gap between letter recognition and practical communication. The temporal aggregation stage is critical for video-based inference, as it eliminates the noise from frame-by-frame predictions and produces stable letter sequences.

The mobile application's dual-mode design addresses two distinct user populations: individuals who need to communicate with sign language users (Free Mode) and individuals who wish to learn sign language (Practice Mode). The real-time feedback mechanism in Practice Mode, with color-coded visual cues and confidence scoring, provides an engaging and effective learning experience.

The TFLite quantization achieves a 9.4$\times$ compression ratio, reducing the model to only 3.03 MB, making it practical for mobile deployment without requiring substantial storage or memory resources.

### B. Limitations

This work has several limitations. First, the dataset is relatively small (2,427 images with approximately 100 samples per class), which may limit generalization to diverse hand shapes, skin tones, lighting conditions, and backgrounds. The small validation set (480 samples) also contributes to noisy validation metrics, making it difficult to precisely gauge model improvement during training. Second, the system handles only static gestures, excluding the dynamic letters J and Z which require motion trajectories. Third, the NLP word formation module relies on dictionary-based greedy segmentation, which may produce incorrect word boundaries for ambiguous letter sequences and cannot handle out-of-vocabulary words. Fourth, the letters W (recall = 0.70), T (recall = 0.80), and U (recall = 0.90) still present recognition challenges, likely exacerbated by the low input resolution of 64$\times$64 pixels which discards fine-grained spatial details needed to distinguish visually similar gestures. Fifth, the current framework does not evaluate post-quantization accuracy degradation; a formal comparison between the H5 and TFLite models on the same test set would strengthen the deployment claims.

### C. Future Work

Several directions can extend this work:

- **Unfreezing the backbone:** Progressive fine-tuning of deeper MobileNetV2 layers may further improve accuracy for challenging gesture pairs such as W/E and T/M/N.
- **Higher resolution inputs:** Increasing input resolution to 128×128 or 224×224 may capture fine-grained spatial features needed to distinguish visually similar gestures.
- **Dynamic gesture recognition:** Integrating LSTM or Transformer-based temporal modules for motion-dependent letters (J, Z) and continuous signing.
- **Advanced NLP:** Incorporating language models such as BERT for context-aware word prediction and auto-correction of recognized letter sequences.
- **Dataset expansion:** Collecting a larger and more diverse dataset with varied skin tones, lighting conditions, and backgrounds to improve generalization and reduce validation variance.
- **Attention mechanisms:** Incorporating spatial attention to focus on discriminative hand regions for ambiguous gestures.
- **Full-integer quantization:** Exploring INT8 quantization with calibration data for further compression and edge TPU acceleration, alongside a formal post-quantization accuracy evaluation.
- **Learning rate scheduling:** Employing cosine annealing or reduce-on-plateau scheduling to mitigate the validation loss instability observed during training.
- **Expanded Practice Mode:** Adding word-level and sentence-level practice, gamification elements, and progress tracking for enhanced learning engagement.

## IX. Conclusion

This paper presented an optimized MobileNetV2-based transfer learning framework for American Sign Language alphabet recognition, integrated with an NLP pipeline for automated text formation and deployed on a dual-mode mobile application. The proposed system achieves 97.29% test accuracy and 98.75% best validation accuracy across 24 static ASL gesture classes, with 15 classes reaching perfect F1-scores. The NLP module converts frame-level letter predictions into meaningful words and sentences through temporal aggregation, dictionary-based word segmentation, and sentence construction. Through TensorFlow Lite quantization, the model size was reduced from 28.45 MB to 3.03 MB a 9.4× compression enabling efficient mobile deployment. The mobile application provides both a Free Mode for real-time fingerspelling recognition with text formation, and a Practice Mode for interactive sign language learning with reference postures, live camera feedback, confidence scoring, and color-coded correctness indicators. This work contributes a practical, end-to-end pipeline for ASL recognition that combines high classification accuracy, NLP-driven text formation, efficient mobile deployment, and an interactive learning interface, addressing critical gaps in the accessibility technology landscape.

## Acknowledgment

The author would like to acknowledge the support of the Indian Institute of Information Technology Kota and the anonymous reviewers for their constructive feedback.

## References

[1] World Health Organization, "Deafness and hearing loss," WHO Fact Sheet, 2023, accessed: 2025-01-15. [Online]. Available: https://www.who.int/news-room/fact-sheets/detail/deafness-and-hearing-loss

[2] R. E. Mitchell, T. A. Young, B. Bachleda, and M. A. Karchmer, "How many people use ASL in the United States? Why estimates need updating," *Sign Language Studies*, vol. 6, no. 3, pp. 306–335, 2006.

[3] A. Wadhawan and P. Kumar, "Deep learning-based sign language recognition system for static signs," *Neural Computing and Applications*, vol. 32, pp. 7957–7968, 2020.

[4] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 2019, pp. 6105–6114.

[5] V. Adithya and R. Rajesh, "A deep convolutional neural network approach for static hand gesture recognition," *Procedia Computer Science*, vol. 171, pp. 2353–2361, 2020.

[6] M. A. Rahaman, K. U. Oyshe, P. K. Chowdhury, T. Debnath, A. Rahman, and M. S. I. Khan, "Computer vision-based six-layered ConvNeural network to recognize sign language for both numeral and alphabet signs," *Biomimetic Intelligence and Robotics*, vol. 4, p. 100141, 2024.

[7] Y. Obi, K. S. Claudio, V. M. Budiman, S. Achmad, and A. Kurniawan, "Sign language recognition system for communicating to people with disabilities," *Procedia Computer Science*, vol. 216, pp. 13–20, 2022.

[8] K. Y. Lum, Y. H. Goh, and Y. B. Lee, "American sign language recognition based on MobileNetV2," *Advances in Science, Technology and Engineering Systems Journal*, vol. 5, no. 6, pp. 1730–1737, 2020.

[9] M. Bani Baker *et al.*, "Image-based Arabic Sign Language recognition system using transfer deep learning models," *Applied Computational Intelligence and Soft Computing*, vol. 2023, p. 5195007, 2023.

[10] S. Dash *et al.*, "Indian sign language recognition using MobileNetV2 fine-tuned by transfer learning," *Computación y Sistemas*, 2024.

[11] N. F. Attia, M. T. F. S. Ahmed, and M. A. M. Alshewimy, "Efficient deep learning models based on tension techniques for sign language recognition," *Intelligent Systems with Applications*, vol. 20, p. 200284, 2023.

[12] S. Jagtap, K. Jadhav, R. Temkar, and M. Deshmukh, "Real-time sign language recognition using MobileNetV2 and transfer learning," *arXiv preprint arXiv:2412.07486*, 2024.

[13] D. R. Kothadiya, C. M. Bhatt, A. Rehman, F. S. Alamri, and T. Saba, "SignExplainer: An explainable AI-enabled framework for sign language recognition with ensemble learning," *IEEE Access*, vol. 11, pp. 47 410–47 419, 2023.

[14] S. Ingoley and J. Bakal, "Indian Sign Language recognition using hand-pose key points and transfer learning," *International Journal of Applied Engineering & Technology*, vol. 5, no. 4, 2023.

[15] B. Karsh, R. H. Laskar, and R. K. Karsh, "mIV3Net: Modified inception V3 network for hand gesture recognition," *Multimedia Tools and Applications*, 2023.

[16] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520.

[17] B. Kang, S. Tripathi, and T. Q. Nguyen, "Real-time sign language fingerspelling recognition using convolutional neural networks from depth map," *IAPR Asian Conference on Pattern Recognition (ACPR)*, 2015.

[18] R. Sharma and V. Kumar, "MobileNetV2-based lightweight real-time Indian Sign Language recognition," *Computers and Electrical Engineering*, vol. 110, p. 108803, 2024.

[19] S. Daga, A. Dusane, and D. Bobby, "With you—Indian Sign Language detection and alert system," in *Proceedings of the International Conference on Emerging Smart Computing and Informatics (ESCI)*, 2024.

[20] Y. Zhang, J. Wang, X. Wang, H. Jing, Z. Sun, and Y. Cai, "Static hand gesture recognition method based on the vision transformer," *Neural Computing and Applications*, 2023.

[21] X. Xu, K. Li, C. Li, and H. R. Guo, "Cnn-lstm for dynamic gesture recognition in hand electrostatic signal processing," in *Proceedings of the 5th International Symposium*. IEEE, 2025.

[22] A. Nunez-Marcos, O. Perez-de Vinaspre, and G. Labaka, "A survey on sign language machine translation," *Expert Systems with Applications*, vol. 213, p. 118993, 2023.

[23] S. Renjith, R. Manazhy, and M. S. Sumi Suresh, "Sign language recognition by using spatio-temporal features," in *Procedia Computer Science*, vol. 233, 2024, pp. 353–362.

[24] K. Sharma, K. A. Aaryan, U. Dhangar, R. Sharma, and S. Taneja, "Automated Indian Sign Language recognition system using LSTM models," in *Proceedings of the IEEE International Conference on Computer Communication and Informatics (ICCCIS)*, 2022, pp. 461–466.

[25] H. ZainEldin, S. A. Gamel *et al.*, "A comprehensive review of AI, DL, and ML technologies for deaf and mute communication," *Artificial Intelligence Review*, 2024.

[26] J. Sunuwar, S. Borah, and A. Kharga, "NSL23 dataset for alphabets of Nepali Sign Language," *Data in Brief*, vol. 53, p. 110080, 2024.

[27] A. A. J. Jim, I. Rafi, M. Z. Akon, U. Biswas, and A. A. Nahid, "KU-BdSL: An open dataset for Bengali Sign Language recognition," *Data in Brief*, vol. 51, 2023.

[28] A. Abdalla, A. Alsereidi, N. Alyammahi, F. B. Qehaizel, H. A. Ignatious, and H. El-Sayed, "An innovative Arabic text sign language translator," in *Procedia Computer Science*, vol. 224, 2023, pp. 425–430.

[29] M. Islam, M. Aloraini, S. Aladhadh, S. Habib, A. Khan, A. Alabdulatif, and T. M. Alanazi, "Toward a vision-based intelligent system: A stacked encoded deep learning framework for sign language recognition," *Sensors*, vol. 23, no. 22, 2023. [Online]. Available: https://www.mdpi.com/1424-8220/23/22/9068

[30] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, 2015.